

From Agent Observability to Governance Evidence: A Privacy-Constrained Telemetry Contract

Imran Siddique
OPAQUE Systems

<https://github.com/agentrust-io/agentrust-telemetry>

October 1, 2026
Technical report, version 2

Abstract

Agent observability conventions describe model, tool, and agent operations, but governance facts are commonly fragmented across policy engines, approval stores, cost modules, and audit systems. Copying those facts into ordinary traces creates two hazards: sensitive payload capture and the false inference that sampled operational telemetry is complete audit evidence. This technical report describes AgenTrust Telemetry, a backend-neutral contract for six governance event families: policy decisions, approval lifecycles, usage, classified data flows, action execution, and evidence lifecycle. The contract correlates with W3C Trace Context and OpenTelemetry without installing a provider, exporter, or competing tracing model. A metadata-only profile rejects prompts, outputs, source code, tool arguments and results, credentials, and authorization tokens by key. Durable run and action identifiers survive process and asynchronous handoffs; propagated metadata remains untrusted and does not confer identity or authority. An optional accumulator accepts events before lossy export and can be finalized into a separately verifiable TRACE record. Its completeness status is a producer assertion that the accumulator records but does not measure. The evaluated release, 0.1.0-alpha.2, ships Python and TypeScript reference SDKs over shared schemas and a portable conformance set of six valid and seven invalid fixtures. At that commit 111 Python unit tests pass and all 13 fixtures produce their expected verdicts, on September 3, 2026 and again when rerun for this edition. A defect found after the evaluation let an edited evidence snapshot be signed; it is reported beside the results it qualifies. The contribution is not another agent tracing convention; it is a narrow semantic boundary between operational observation and governance evidence whose completeness must be stated rather than inferred.

Scope of this edition. This is an author-prepared technical report, not peer reviewed and not a normative specification. Version 2 (October 1, 2026) corrects two errors in version 1 (September 27, 2026): the statement that approval digests prevent an approval from being attached to another action, which the reference finalizer does not check, and a typesetting fault that dropped every bold face from the published PDF; results are unchanged. Version 1 revised a manuscript dated September 3, 2026. The contract it describes is alpha and may change incompatibly; current behavior, normative schemas, and limitations are maintained in the project repository. The Python unit suite, the portable conformance runner, and the repository’s cross-runtime contract checks were rerun on September 27, 2026 at the evaluated commit (29c13cf, release 0.1.0-alpha.2). The TypeScript test suite was not rerun. Findings published after the evaluation are reported in Section 7.

Patent Pending.

1 Introduction

An agent run can cross model providers, tool servers, queues, approval systems, policy engines, databases, and human decision points. OpenTelemetry (OTel) supplies a broadly adopted model for traces, metrics, logs, events, baggage, and context propagation. Its generative-AI conventions, now maintained in a dedicated repository with development status, describe model, tool, and agent operations [OpenTelemetry Authors, 2026b]. OpenInference similarly defines

AI-observability span conventions [Arize AI, 2026]. Those systems answer operational questions: what ran, how long it took, where it failed, and how calls relate.

Governance asks different questions. Which policy and enforcement mode produced a decision? Was approval requested before execution, and was the executed action the one approved? Which cost source produced an amount? Did classified data move between named trust zones? Was evidence accumulation complete before finalization? These facts may appear in proprietary logs, but inconsistent names and identifiers prevent portable analysis.

A second problem is semantic, not syntactic. Telemetry pipelines are routinely sampled, batched, retried, transformed, and dropped. An audit record that claims complete execution evidence cannot be reconstructed from a backend query unless completeness was established before the lossy boundary. AgenTrust Telemetry therefore treats OTel projection and durable evidence as two consumers of a validated event, not as interchangeable representations.

2 Requirements and non-goals

The contract has five requirements:

1. remain independent of agent framework, policy engine, collector, and storage backend;
2. reuse OTel context and fields where their semantics already fit;
3. correlate governance events across processes and asynchronous handoffs;
4. reject raw or secret-bearing content in the metadata-only profile;
5. make evidence completeness an explicit producer claim rather than an inference from observed events.

The project is not a collector, dashboard, tracing backend, policy evaluator, approval workflow, pricing catalog, or agent auto-instrumentation library. It does not duplicate spans already produced by OTel GenAI or OpenInference instrumentation. An event reports a fact observed by another component; it does not authorize an action merely by existing.

3 Event contract

Six event families provide a common envelope and family-specific fields.

Family	Governance fact
Policy decision	allow, deny, challenge, or error; enforcement mode; policy identity; latency
Approval lifecycle	request, decision, expiry, cancellation, and execution outcome bound to an action digest
Usage	input/output units, cost, currency, scope, and provenance of measurement or estimate
Data flow	classified source-to-destination metadata without payload capture
Action execution	resolved tool, MCP, A2A, file, HTTP, or database attempt and outcome
Evidence lifecycle	checkpoint, accumulation state, completeness declaration, and optional TRACE finalization

Table 1: Normalized governance event families.

Every event carries a durable `run_id`, an `agent_id`, an event time encoded as a decimal string of nanoseconds so that no runtime loses precision, and when applicable a stable `action_id` and canonical action digest. `trace_id` and `span_id` correlate the event with operational context but do not replace durable identifiers. This matters when one logical run spans multiple traces, waits in a queue, or must be verified after the tracing backend’s retention period.

Usage events distinguish direct measurements from estimates and distinguish an individual operation from a rollout. A cost value without currency and provenance is not portable. Approval events carry the action digest of the approved intent, so a consumer can detect an approval attached to an action with different arguments. The reference finalizer

does not make that comparison: at the evaluated release, and on the main branch on October 1, 2026, a run whose executed action carries a digest other than the approved one is still appraised as affirming. Data-flow events record classifications, logical sources, destinations, and applicable policy labels without recording the content that crossed the boundary.

All six families are defined by the schemas. At the evaluated release, however, neither SDK provides a helper that emits `approval.cancelled`, the approval execution outcomes, or the `evidence.*` lifecycle events; a producer must construct those events directly.

4 Privacy and propagation

The metadata-only profile prohibits raw prompts, model outputs, source code, tool arguments, tool results, credentials, and authorization tokens. Schema validation is supplemented by recursive key inspection because permissive extension objects could otherwise tunnel prohibited fields, and extension attributes are rejected unless each key is explicitly allowlisted. This is a minimization boundary, not a proof that arbitrary metadata cannot be identifying: a sensitive value placed under an allowed key is not detected.

For synchronous cross-process calls, the SDK injects W3C trace context plus durable AgenTrust identifiers. A receiver extracts the OTel context and uses it as a remote parent. For queue handoffs, the receiver starts a new trace and links it to the extracted context, preserving causality without misrepresenting queued work as a synchronous child span. Propagated identifiers are untrusted input: they establish neither workload identity nor authorization.

W3C Trace Context standardizes trace propagation [Kanzhelev et al., 2024]; it intentionally does not provide identity or security. AgenTrust Telemetry follows that boundary and requires authoritative identity and policy systems to emit their own verified facts.

5 Projection and evidence finalization

The normalized JSON event is the source contract. OTel is a projection. Standard OTel attributes take precedence, and AgenTrust extensions are added only for governance semantics not represented by the pinned convention. Trace and span identifiers remain native context rather than duplicated string attributes. Event projection follows OTel’s guidance that events represent point-in-time occurrences or state transitions within a longer operation [OpenTelemetry Authors, 2026a].

The SDK can deliver a validated event to two independent sinks. An OTel emitter supports operational search and metrics. An evidence accumulator receives the event before sampling or export and maintains an ordered SHA-256 chain over RFC 8785 canonical bytes of accepted events. It records a caller-supplied completeness label; it does not independently measure whether every source event was captured. Finalization may produce a TRACE record containing the chain commitment, policy and approval summaries, usage totals with provenance, data-flow classifications, and the completeness status [Poddar et al., 2026].

With the reference accumulator configured as the evidence sink, each operational projection attempted by an invocation follows successful evidence acceptance. In durable callback mode, acceptance additionally follows the callback’s acknowledgment, and a refused or failed acknowledgment raises before any projection is attempted. A projection that fails after acceptance is reported separately and does not remove the accepted event. This is an ordering property, not a completeness property.

A complete-capture claim needs more than that ordering: an independently defined set of required events, instrumentation at every declared boundary, and a recovery policy. The accumulator verifies none of these. An empty accumulator can be sealed as complete, so the label alone proves neither that required events were emitted nor that they were retained; the TRACE finalizer at the evaluated release refuses open, incomplete, or empty evidence and marks its output `software-only`. A durable callback must supply its own storage, idempotency, and recovery: if an external store commits but its acknowledgment is lost, a retry repeats the external write. A backend query cannot establish these properties after the fact. TRACE supplies a signed evidence container and verification semantics; AgenTrust Telemetry supplies the event normalization and pre-export accumulation boundary.

6 Implementation and conformance

The project publishes JSON Schema 2020-12 contracts, Python and TypeScript SDKs, valid and invalid fixtures, and an independent Python conformance runner. Both SDKs consume the same schema corpus, and a repository check compares the TypeScript schema copies byte for byte with the normative files. TypeScript preserves nanosecond timestamps as decimal strings to avoid JavaScript numeric truncation. Neither SDK installs a global OTel provider, processor, exporter, meter, or propagator; callers retain operational ownership.

The portable fixture corpus includes six valid examples, one for each family, and seven invalid examples. Negative cases reject raw content, an all-zero trace identifier, numeric timestamp ambiguity, usage without a measurement basis, a model call incorrectly marked as a rollout, aggregate cost without rollout semantics, and action execution without an action digest. Repository tests cover policy adapters, approval state, AGT audit and data projection, OTel projection, propagation, evidence accumulation, TRACE finalization, cross-language schema synchronization, and repository gates. The checked OTel projection matrix, pinned to a development-status commit of the GenAI conventions captured on August 18, 2026, covers 16 span attributes, eight metrics, and six event families.

Results. The evaluated commit is 29c13cf (release 0.1.0-alpha.2, the repository’s main branch on September 3, 2026). On that date 111 Python unit tests passed and all 13 portable fixtures produced their expected results. A rerun at the same commit on September 27, 2026 reproduced both counts: 111 tests passed with no skips, the runner reported six valid and seven invalid fixtures as expected, the projection matrix check reported 16 span attributes, eight metrics, and six event families, and the TypeScript schema copies matched the normative bytes. The rerun pinned the optional TRACE dependency to agentrust-trace 0.9.0, the release current on September 3. With the version resolved on September 27 (0.11.0), one test fails: agentrust-trace 0.10.0 began rejecting a non-Ed25519 stand-in signing key that the test used, which the repository corrected in its test on September 6. Four scripted probes of the evaluated code confirmed the ordering and completeness statements in the previous section: a refused acknowledgment raised before any log projection; a failing log exporter left the accepted event in place and was reported as a projection error; an empty accumulator sealed as complete; and a lost acknowledgment followed by a retry produced two external writes and one local entry. For comparison, the current main branch (release 0.1.0-alpha.5) passes 128 Python tests with the same 13 fixtures and the same matrix counts.

7 Findings after the evaluation

Two changes merged after September 3, 2026 narrow the results above.

Finalization did not recheck the evidence chain. At the evaluated release, TRACE finalization took the runtime measurement from the snapshot’s chain digest and derived the appraisal from the snapshot’s entries without checking that the entries still hashed to that digest. Because the snapshot holds mutable objects, a caller could seal a run holding a deny decision, edit the entry to allow, and obtain a signed record affirming the edited decision under a measurement that commits to the original. The fix, merged on September 25, 2026 and released in 0.1.0-alpha.5, recomputes the chain before signing in both SDKs and refuses on mismatch. Run against the evaluated code, the four regression tests added with that fix produce three failures (an edited event, a dropped entry, and a substituted chain digest are all signed) and one error. The finalization results in the previous section therefore do not extend to snapshots that can be modified between sealing and signing.

Validation and locking edge cases. The same change found that validation at the evaluated release raised an uncaught recursion error on deeply nested input, raised on non-string keys, and accepted NaN and infinite numbers until the evidence chain rejected them; all three now fail validation. A change merged on September 6, 2026 stopped the Python accumulator from deadlocking when a durable callback re-entered it on the same thread.

8 Related work and novelty boundary

Observability conventions. OTel defines the telemetry data model, context propagation, and event guidance [OpenTelemetry Authors, 2026a]. Its GenAI conventions define agent, tool, and inference spans and treat message content as opt-in attributes [OpenTelemetry Authors, 2026b]. OpenInference defines span attributes such as `input.value` and

output.value that carry payloads unless hiding options are configured [Arize AI, 2026]. W3C Trace Context provides interoperable propagation [Kanzhelev et al., 2024], and CloudEvents standardizes event envelopes across systems [CloudEvents Authors, 2022]. AgenTrust Telemetry composes with these surfaces and does not replace them.

Governance telemetry. Pathak and Jain’s Governance-Aware Agent Telemetry (GAAT) is the closest prior work [Pathak and Jain, 2026]. It defines a Governance Telemetry Schema that extends OTel with governance attributes such as classification, jurisdiction, and sensitivity; evaluates OPA-compatible rules with sub-200 ms enforcement; routes graduated interventions through an enforcement bus; and adds a Trusted Telemetry Plane with ECDSA P-256 signed events, a Merkle-tree audit log, replay detection, and statistical omission detection trained on expected event sequences. Its evaluation reports violation prevention rates on synthetic and trace-derived workloads. GAAT therefore already covers governance attributes on OTel, policy evaluation over telemetry, and cryptographic provenance of telemetry. Kulkarni’s Governance-Aware Observability Pipeline (GAOP) enforces compliance rules, including PII redaction, inside the telemetry pipeline and adds cryptographic lineage [Kulkarni, 2025]. Policy engines and approval systems also expose their own decision logs.

Tamper-evident agent records. Hash-chained and Merkle-structured audit logs are long established [Schneier and Kelsey, 1999, Crosby and Wallach, 2009]. The Agent Audit Trail Internet-Draft, first posted in March 2026, specifies JSON agent audit records linked by SHA-256 hash chaining over RFC 8785 canonical bytes, with optional signatures and, in later revisions, a record phase that distinguishes pre-execution from post-execution records [Sharif, 2026]. Two reports posted in September 2026, after the repository was public, are concurrent with this work: Agent Flight Recorder adds Merkle batching and on-chain anchoring to canonical agent-action events [Bindschaedler et al., 2026], and NovaFabric seals agent runs into signed, replayable capsules built on OTel and measures declared-stream completeness [Ardebili, 2026].

What this report does not claim. It does not claim governance event families, governance attributes on OTel, policy enforcement from telemetry, payload redaction, hash-chained or signed audit records, or a measured completeness guarantee. Governance event families alone are not novel.

Narrow delta. The remaining contribution is the conjunction of (1) stable run and action joins across synchronous and queued handoffs, independent of trace identifiers; (2) metadata-only exclusions enforced by the producer SDK before any export, instead of redaction in a downstream pipeline; (3) explicit measurement and cost provenance; and (4) a stated separation between lossy operational export and governance evidence, in which events are accepted before export and completeness is an explicit producer assertion carried into a separately verifiable TRACE record, never inferred from a backend. The claim would be narrowed further by a prior cross-runtime convention that supplies the same producer-side exclusions, durable joins, and separation of export from evidence.

9 Limitations and conclusion

The contract is alpha and may change. Schemas constrain representation, not the truth of producer observations. Metadata can still be sensitive. Coverage depends on instrumentation at every declared governance boundary. An accumulator cannot observe bypassed code, and a signature cannot repair an incomplete capture. The accumulator’s memory mode is not durable, and in callback mode the adopter owns storage, idempotency, and recovery. Interoperability was exercised locally with OpenTelemetry Python, not across collectors, backends, or independent implementations, and no performance measurement is reported.

AgenTrust Telemetry’s central result is a boundary: operational traces may be rich but lossy, while a complete-capture claim requires independently justified instrumentation and coverage in addition to accumulation before loss. A small event contract, shared schemas, privacy failures, durable joins, and optional TRACE finalization make that boundary portable without creating another tracing backend.

References

Mohsen Seyedkazemi Ardebili. NovaFabric: Tamper-evident, replayable evidence for autonomous AI agent runs. arXiv:2609.12582, 2026. URL <https://arxiv.org/abs/2609.12582>.

- Arize AI. OpenInference specification, 2026. URL <https://github.com/Arize-ai/openinference/tree/main/spec>.
- Laurent Bindschaedler, Quentin Botha, and Christoph Siebenbrunner. Agent Flight Recorder: Tamper-evident audit trails with on-chain anchoring for long-horizon tool-using agents. arXiv:2609.01931, 2026. URL <https://arxiv.org/abs/2609.01931>.
- CloudEvents Authors. CloudEvents specification, version 1.0.2. Cloud Native Computing Foundation, 2022. URL <https://github.com/cloudevents/spec>.
- Scott A. Crosby and Dan S. Wallach. Efficient data structures for tamper-evident logging. In *Proceedings of the 18th USENIX Security Symposium*, 2009. URL <https://www.usenix.org/conference/usenixsecurity09/technical-sessions/presentation/efficient-data-structures-tamper-evident>.
- Sergey Kanzhelev, Daniel Dyla, Yuri Shkuro, J. Kalyana Sundaram, and Bastian Krol. Trace Context level 2. Candidate recommendation draft, W3C, 2024. URL <https://www.w3.org/TR/2024/CRD-trace-context-2-20240328/>.
- Priyanka Kulkarni. Governance-Aware Observability Pipeline (GAOP): Embedding compliance enforcement and cryptographic lineage into telemetry data flows. *International Journal of Computer Applications*, 187(50), 2025. doi: 10.5120/ijca2025925867.
- OpenTelemetry Authors. Semantic conventions for events. Development status, 2026a. URL <https://opentelemetry.io/docs/specs/semconv/general/events/>.
- OpenTelemetry Authors. OpenTelemetry GenAI semantic conventions. Development status; the evaluated compatibility matrix pins commit a685613, 2026b. URL <https://github.com/open-telemetry/semantic-conventions-genai>.
- Anshul Pathak and Nishant Jain. Governance-Aware Agent Telemetry for closed-loop enforcement in multi-agent AI systems. arXiv:2604.05119, 2026. URL <https://arxiv.org/abs/2604.05119>.
- Rishabh Poddar, Aaron Fulkerson, and Imran Siddique. TRACE: Trust, runtime attestation, and compliance evidence (A portable attestation format for AI agent runtime governance). Technical report, version 1, OPAQUE Systems, 2026. URL <https://agenttrust-io.com/research/trace/v1/>.
- Bruce Schneier and John Kelsey. Secure audit logs to support computer forensics. *ACM Transactions on Information and System Security*, 2(2), 1999. doi: 10.1145/317087.317089.
- R. Sharif. Agent Audit Trail: A standard logging format for autonomous AI systems. Internet-Draft draft-sharif-agent-audit-trail-05, IETF, first posted in March, 2026. URL <https://datatracker.ietf.org/doc/draft-sharif-agent-audit-trail/>.